

DJANGO FRAMEWORK

LIBRARIES

PYTHON LIBRARIES

1. Python Collections - Container Datatypes:

- **Purpose:** Provides specialized container datatypes that support efficient handling of data.
- **Key Types:**
 - **List:** Ordered, mutable, allows duplicates.
 - **Tuple:** Ordered, immutable, allows duplicates.
 - **Set:** Unordered, no duplicates, fast membership testing.
 - **Dictionary:** Unordered, key-value pairs, fast lookups.
- **Common Use:** Data manipulation, storing and accessing collections of data in web apps (like user data or API responses).

2. Tinker

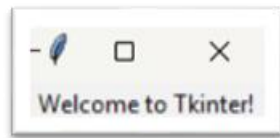
- **Purpose:** Python's standard library for creating graphical user interfaces (GUIs).
- **Keyfeatures:**
 - Widgets: Buttons, labels, text boxes, etc.
 - Event handling: Respond to user interactions like clicks or key presses.
 - Simple layout management.

Code:

```
from tkinter import Tk, Label
# Create a window
root = Tk()
root.title("Hello Window")
# Add a label to display text
Label(root, text="Welcome to Tkinter!").pack()
# Run the application
```

```
root.mainloop()
```

Output:

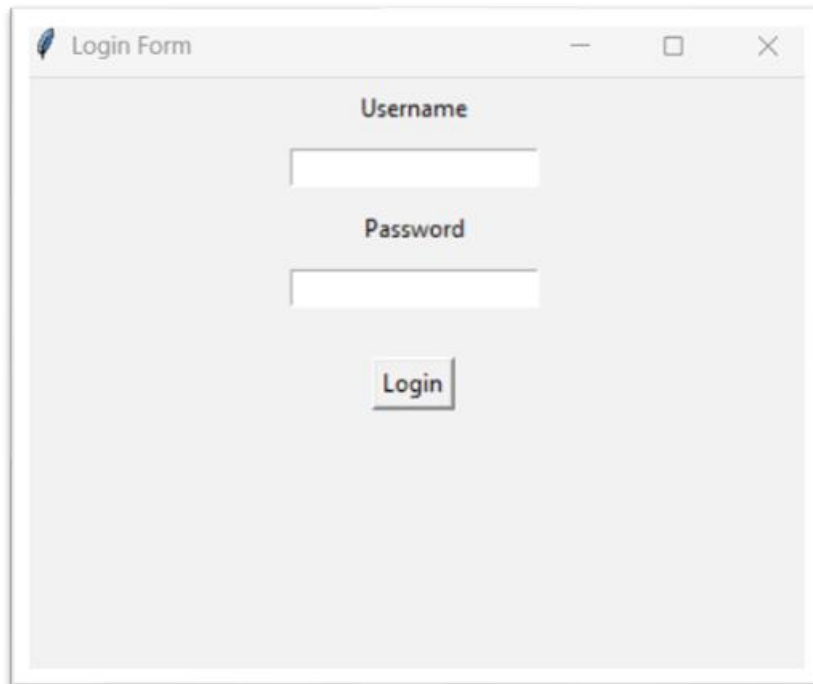


3.Requests - HTTP Requests:

- **Purpose:** Simplifies HTTP requests to interact with web APIs.
- **Key Features:**
 - Send GET, POST, PUT, DELETE requests easily.
 - Handle request parameters, headers, and cookies.
 - Simple error handling and response handling.
- **Common Use:** Interact with REST APIs, download content from the web.

Code:

```
from tkinter import Tk, Label, Entry, Button
def login():
    username = username_entry.get()
    password = password_entry.get()
    print(f'Username: {username}, Password: {password}') # Placeholder for real
    login logic
# Create main window
root = Tk()
root.title("Login Form")
root.geometry("300x200") # Set size of the window
# Username Label and Entry
Label(root, text="Username", font=('Arial', 10, 'bold')).pack(pady=(10, 0))
username_entry = Entry(root, width=30)
username_entry.pack(pady=(5, 10))
# Password Label and Entry
Label(root, text="Password", font=('Arial', 10, 'bold')).pack()
password_entry = Entry(root, show="*", width=30)
password_entry.pack(pady=(5, 10))
# Login Button
Button(root, text="Login", width=10, command=login).pack(pady=10)
# Run the application
root.mainloop()
```

Output:

The image shows a web browser window with the title 'Login Form'. Inside the window, there is a login form with the following elements:

- A label 'Username' above a text input field.
- A label 'Password' above a text input field.
- A 'Login' button centered below the password field.

4.Scrapy:

- **Purpose:** An open-source web crawling framework for large-scale web scraping.
- **Key Features:**
 - Fast, extensible, and asynchronous web scraping.
 - Supports handling requests, data extraction, and storing results.
 - Built-in handling for logging, retries, and sessions.
- **Common Use:** Web crawling and scraping projects that require high performance.

5.BeautifulSoup4 - Web Scraping:

- **Purpose:** Parses HTML and XML documents to extract data.
- **Key Features:**
 - Easy navigation and searching within HTML.
 - Supports different parsers like `html.parser`, `lxml`, and `html5lib`.
- **Common Use:** Extract data from websites for analysis, e.g., for building data-driven applications

Code:

```

import requests
from bs4 import BeautifulSoup
def scrape_quotes():
    base_url = "http://quotes.toscrape.com"
    next_page = "/"
    while next_page:
        response = requests.get(base_url + next_page)
        if response.status_code == 200:
            soup = BeautifulSoup(response.text, "html.parser")
            quotes = soup.find_all("span", class_="text")
            authors = soup.find_all("small", class_="author")
            for quote, author in zip(quotes, authors):
                print(f"{quote.text}" - {author.text}\n')
            next_btn = soup.find("li", class_="next")
            next_page = next_btn.a["href"] if next_btn else None
        else:
            print(f"Failed to fetch webpage. Status code: {response.status_code}")
            break
    scrape_quotes()

```

Output:

```

(myenv) C:\Users\Lenovo>python -u "c:\Users\Lenovo\import requests.py"

""The world as we have created it is a process of our thinking. It cannot be changed without
changing our thinking."" - Albert Einstein

""It is our choices, Harry, that show what we truly are, far more than our abilities."" - J.K. Rowling

""There are only two ways to live your life. One is as though nothing is a miracle. The other is as
though everything is a miracle."" - Albert Einstein

""The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably
stupid."" - Jane Austen

""Imperfection is beauty, madness is genius and it's better to be absolutely ridiculous than
absolutely boring."" - Marilyn Monroe

""Try not to become a man of success. Rather become a man of value."" - Albert Einstein

""It is better to be hated for what you are than to be loved for what you are not."" - André Gide

""I have not failed. I've just found 10,000 ways that won't work."" - Thomas A. Edison

""A woman is like a tea bag; you never know how strong it is until it's in hot water."" - Eleanor
Roosevelt

```

"“A day without sunshine is like, you know, night.”" - Steve Martin

6.Zappa:

- **Purpose:** Deploy Python web applications to AWS Lambda and API Gateway.
- *Key Features:*
 - Supports frameworks like Flask and Django for serverless deployments.
 - Manages serverless architecture and deployment configurations.
- **Common Use:** Build scalable, serverless web apps without maintaining servers.

7.Dash:

- **Purpose:** Web application framework for building interactive data visualization applications.
- *Key Features:*
 - Built on top of Flask, React, and Plotly.
 - Integrates seamlessly with data science libraries (e.g., Pandas, Plotly).
- **Common Use:** Building dashboards and data-driven web applications.

Turbo Gears

- **Purpose:** Full-stack web framework built on top of WSGI.
- *Key Features:*
 - Modular: Mix and match components like SQLAlchemy, Genshi, and others.
 - Focus on rapid development and scalability.
- **Common Use:** Develop scalable, enterprise-level web applications.

8.CherryPy:

- **Purpose:** Minimalistic web framework for building web applications.
- *Key Features:*
 - Provides a simple and fast HTTP server.
 - Handles routing, cookies, sessions, and file uploads.

- **Common Use:** Building web applications with a lightweight framework.

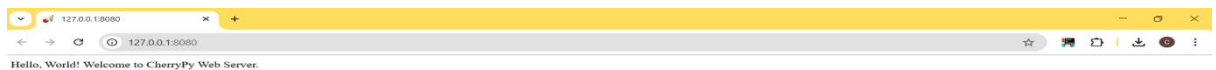
Code:

```
import cherrypy
class HelloWorld:
    @cherrypy.expose # Exposes this method as a web page
    def index(self):
        return "Hello, World! Welcome to CherryPy Web Server."
# Configure and start the CherryPy server
if __name__ == "__main__":
    cherrypy.quickstart(HelloWorld(), "/", config={
        "global": {
            "server.socket_host": "127.0.0.1", # Localhost
            "server.socket_port": 8080,      # Port number
        }
    })
```

Output:

```
(myenv) C:\Users\Lenovo>python -u "c:\Users\Lenovo\import requests.py"
[10/Apr/2025:01:34:09] ENGINE Listening for SIGTERM.
[10/Apr/2025:01:34:09] ENGINE Bus STARTING
[10/Apr/2025:01:34:09] ENGINE Started monitor thread 'Autoreloader'.
[10/Apr/2025:01:34:09] ENGINE Serving on http://127.0.0.1:8080
[10/Apr/2025:01:34:09] ENGINE Bus STARTED
```

After run the server :-



Django: A Web Framework for Python

Django is a high-level Python web framework that empowers developers to build secure, scalable, and maintainable web applications quickly and efficiently. It follows the Model-View-Template (MVT) architectural pattern, promoting rapid development and clean, pragmatic design.

Key Features of Django:

- **Rapid Development:**

Django comes with built-in features such as authentication, database management, and an auto-generated admin panel, which speed up the development process.

- **Scalability:**

Designed to scale seamlessly, Django is suitable for projects ranging from small websites to enterprise-level applications.

- **Robust Security:**

With built-in protection against common security threats—like SQL injection, CSRF, and XSS—Django helps you build secure web applications.

- **Powerful ORM (Object-Relational Mapper):**

Django's ORM enables developers to interact with the database using Python code instead of writing raw SQL, simplifying database operations.

- **Built-in Admin Interface:**

Django automatically generates an admin dashboard for managing application data, reducing the need for custom backend tools.

- **Modular and Reusable Components:**

The framework encourages a modular design, allowing developers to create reusable apps that can be easily integrated into multiple projects.

❖ Django's MVT Architecture:

- ◆ **Model (M):**

Manages database interactions. It defines the data structure and business logic, representing data objects such as Users, Bookings, and more.

- ◆ **View (V):**

Handles the business logic. It retrieves data from models and sends it to the appropriate template for rendering.

◆ **Template (T):**

Responsible for presenting data. Templates dynamically generate HTML pages based on the data provided by the view

❖ **Example MVT Folder Structure in Django:**

```
myproject/      # Project Directory
├── myproject/  # Project Settings Directory
│   ├── __init__.py
│   ├── settings.py # Project settings
│   ├── urls.py    # URL routing
│   ├── wsgi.py    # WSGI entry point
│   └── asgi.py    # ASGI entry point
├── myapp1/     # Django App Directory
│   ├── migrations/ # Database migrations
│   ├── __init__.py
│   ├── admin.py   # Admin panel configurations
│   ├── apps.py    # App configuration
│   ├── models.py  # Models (Database structure)
│   ├── views.py   # Business logic
│   ├── urls.py    # App-specific URLs
│   └── templates/ # Template folder
│       └── home.html # HTML file for rendering UI
└── manage.py     # Django command-line utility
```


Installation procedure Of Django:

- Install Python (if not installed):

```
python --version
```

- Create a Virtual Environment:

```
python -m venv myenv
```

- Activate the Virtual Environment:

- On Windows:

```
myenv\Scripts\activate
```

- On Mac/Linux:

```
source myenv/bin/activate
```

- Install Django Inside the Virtual Environment:

```
pip install Django
```

To verify the installation, run:

```
django-admin --version
```

- Create a New Django Project:

```
django-admin startproject myproject
```

- Navigate into the Project Directory:

```
cd myproject
```

- Run the Development Server:

```
python manage.py runserver
```

If everything is set up correctly, you will see output like:

```
Starting development server at http://127.0.0.1:8000/
```

Connecting Views and URLs:

Create URL Patterns for Your App :

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
```

```
    path("", views.home, name='home'), # The empty string ("") represents the root URL for
    the app.
```

App's URLs in the Project's URL Configuration

```
from django.contrib import admin
```

```
from django.urls import path, include
```

```
urlpatterns = [
```

```
    path('admin/', admin.site.urls),
```

```
    path("", include('myapp.urls')), # Include the URL patterns from myapp
```

```
]
```

Test Your Setup

1. Create a Template:

Create a simple `home.html` template in your app's templates directory (e.g., `myapp/templates/home.html`):

```
2. html
3. CopyEdit
4. <!-- myapp/templates/home.html -->
5. <!DOCTYPE html>
6. <html lang="en">
7. <head>
8.     <meta charset="UTF-8">
9.     <title>Home</title>
10. </head>
11. <body>
12.     <h1>Welcome to the Home Page!</h1>
13. </body>
14. </html>
```

15. Run the Development Server:

Start your Django development server:

```
python manage.py runserver
```

16. Access the Home Page:

Open your browser and navigate to `http://127.0.0.1:8000/`. You should see your home page rendered by the `home` view.

web application, controlling how data is processed and which HTML templates are displayed.

- **View:** Write a function in `views.py` that handles the request.
- **App URLs:** Map URLs to views in your app's `urls.py`.
- **Project URLs:** Include your app's URLs in the project-level `urls.py`.
- **Template:** Create an HTML template that the view will render.

Views.py:

In Django, views.py is the file where you define functions or classes that handle requests and return responses. Views act as the logic layer of a Django web application, controlling how data is processed and which HTML templates are displayed.

```
from django.shortcuts import render, redirect
from django.contrib import messages
from django.core.mail import send_mail
from django.contrib.auth.decorators import login_required
from .models import Event, EventRegistration
from .forms import ContactForm, EventRegistrationForm
from django.contrib.auth import logout

# Home Page
def home(request):
    return render(request, 'home.html')

# About Page
def about(request):
    return render(request, 'about.html')

# Events Page (Show all events)
def events(request):
    all_events = Event.objects.all()
    return render(request, 'events.html', {'events': all_events})

# Sponsors Page
def sponsors(request):
    return render(request, 'sponsors.html')

# Specific Event Pages
def conference(request):
    return render(request, 'conference.html')
def ai_workshop(request):
    return render(request, 'ai.html')
def chatbot_workshop(request):
    return render(request, 'chatbot.html')

# Contact Page (Form Submission)
def contact(request):
    if request.method == "POST":
        form = ContactForm(request.POST)
        if form.is_valid():
            form.save()
            send_mail(
                "New Contact Form Submission",
                f"Message
from
{form.cleaned_data['name']}: \n{form.cleaned_data['message']}",
                form.cleaned_data['email'],
                ['admin@example.com'],
                fail_silently=False,
            )
            messages.success(request, "Your message has been sent!")
            return redirect('contact')
    else:
```

```

        form = ContactForm()
        return render(request, 'contact.html', {'form': form})
# Event Registration Page
def register_event(request, event_id):
    return redirect('dashboard') # Redirects to the dashboard immediately
# Handle Event Registration Submission
from django.shortcuts import render, redirect
from django.views.decorators.csrf import csrf_exempt
@csrf_exempt # Temporarily disable CSRF for testing (Not recommended for
production)
def submit_registration(request):
    if request.method == "POST":
        # Get form data (you can save it to a database if needed)
        name = request.POST.get("name")
        email = request.POST.get("email")
        phone = request.POST.get("phone")

        # Add logic to save registration details if necessary
        print(f"New registration: {name}, {email}, {phone}")

        # Redirect to the dashboard after successful submission
        return redirect("dashboard") # Ensure 'dashboard' is defined in
urls.py

    return render(request, "register.html") # Reload form if not a POST
request
# ✓ Redirect to Dashboard after successful registration
    return redirect('dashboard')
    messages.error(request, "Invalid request method.")
    return redirect('events')
# User Dashboard (Requires Login)
@login_required
def dashboard_view(request):
    if request.user.is_staff:
        return redirect('admin_dashboard')
    else:
        return redirect('user_dashboard')
@login_required
def admin_dashboard(request):
    return render(request, 'dashboard/admin_dashboard.html')
@login_required
def user_dashboard(request):
    return render(request, 'dashboards/user_dashboard.html')
def registration(request):
    return render(request, "registration.html")
def logout_view(request):
    if request.method == 'POST':
        logout(request)
        return render(request, 'logout.html')

```

Creating urls.py in a Django App:

In Django, the urls.py file is responsible for mapping URLs to views. It acts as the router of your application, directing user requests to the correct function in views.py.

Each Django app should have its own urls.py file to define app-specific routes.

Steps to Create urls.py in a Django App

- Inside your Django app folder (myapp1), create a file named **urls.py**.
- Define URL patterns to map URLs to views.

```
# myapp/urls.py
```

```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
```

```
    # Home page of the app
```

```
    path("", views.home, name='home'),
```

```
    # Additional URL patterns:
```

```
    path('about/', views.about, name='about'),
```

```
    path('contact/', views.contact, name='cont']
```

App Urls.Py :

```

from django.urls import path, include
from django.contrib.auth.views import LoginView
from . import views
urlpatterns = [
    path('', views.home, name='home'),
    path('about/', views.about, name='about'),
    path('sponsors/', views.sponsors, name='sponsors'),
    path('contact/', views.contact, name='contact'),
    path('register/<int:event_id>/', views.register_event,
name='register_event'),
    path('registration/', views.registration,
name='registration'),
    path('submit-registration/', views.submit_registration,
name='submit_registration'),
    path('conference/', views.conference, name='conference'),
    path('ai/', views.ai_workshop, name='ai_workshop'),
    path('chatbot/', views.chatbot_workshop,
name='chatbot_workshop'),
    path('events/', views.events, name='events'),
    # Use Django's LoginView with custom template
    path('accounts/login/',
LoginView.as_view(template_name='login.html'), name='login'),
    # You can still include auth URLs if needed (password reset,
etc.)
    path('accounts/', include('django.contrib.auth.urls')),
    path('dashboard/', views.dashboard, name='dashboard'),
    path('logout/', views.logout_view, name='logout'),
    path('user/dashboard/', views.user_dashboard,
name='user_dashboard'),
    path('admin/dashboard/', views.admin_dashboard,
name='admin_dashboard'),
]

```

Templates:

Templates in Django are HTML files enhanced with Django's templating language that dynamically generate content based on the data provided by views. They effectively separate the presentation layer (frontend/UI) from the backend logic, adhering to Django's MVT (Model-View-Template) architectural pattern.

Where to Store Templates?

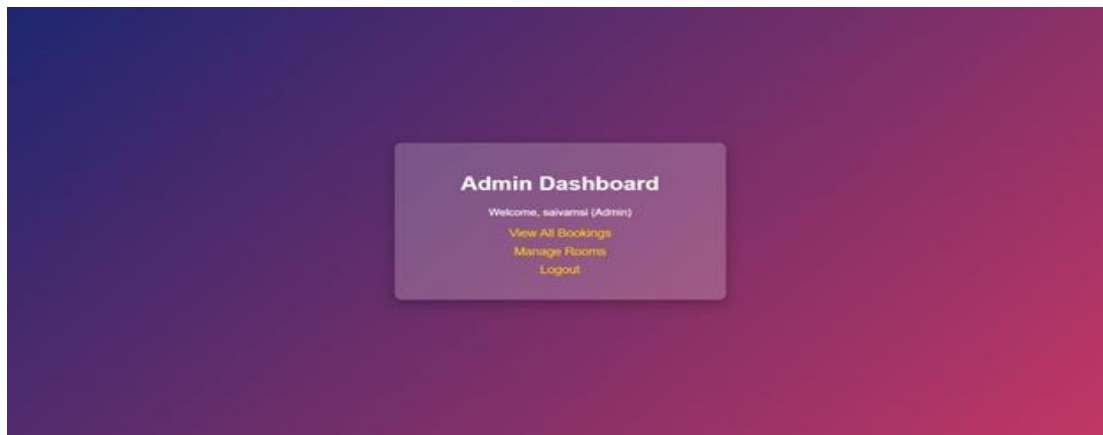
Templates in Django can be stored in different locations based on your project structure and preferences.

```
mypro/
├── mypro/
│   └── ...
├── myapp/
│   ├── migrations/
│   ├── static/
│   │   ├── css/
│   │   ├── js/
│   │   └── images/
│   └── templates/
│       ├── base.html
│       ├── contact.html
│       ├── ai.html
│       ├── chat_bot.html
│       ├── conference.html
│       ├── logout.html
│       ├── login.html
│       ├── dashboard.html
│       ├── event.html
│       ├── home.html
│       └── view_schedule.html
│   ├── admin.py
│   ├── apps.py
│   ├── forms.py
│   ├── models.py
│   ├── urls.py
│   └── views.py
├── manage.py
└── db.sqlite3
```


Admin_dashboard.html :

```
{% extends "base.html" %}
{% block content %}
<div class="dashboard-container">
  <h2 class="glow-text">Admin Dashboard</h2>
  <p>Hello, {{ user.username }} (Admin)</p>
  <!-- Add admin-specific content like event management
links -->
{% endblock %}
```

OUTPUT:



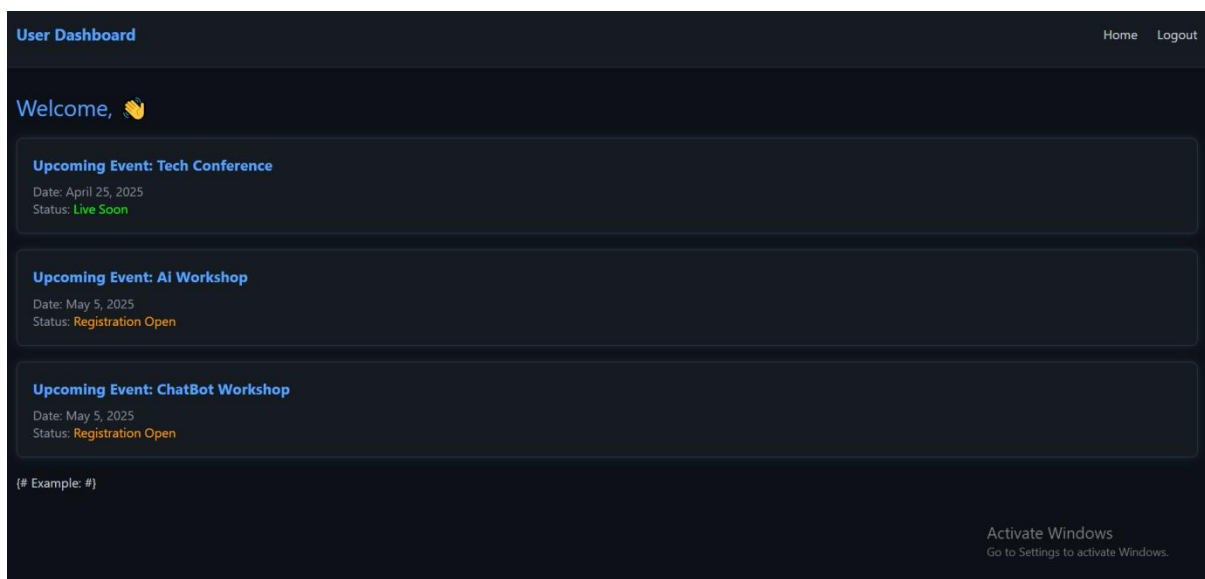
Role of Admin_dashboard.html in Django:

- **Main interface for administrators**, providing access to system-level controls and event management features.
- **Displays admin-specific data**, such as total users, upcoming events, pending approvals, and system notifications.
- **Includes administrative navigation**, with links to manage events, users, sponsors, reports, and settings.
- **Enables approval workflows**, like reviewing event requests, verifying users, or updating statuses.

User Dashboard.html:

```
{% extends "base.html" %}
{% block content %}
<div class="dashboard-container">
    <h2 class="glow-text">User Dashboard</h2>
    <p>Welcome, {{ user.username }}!</p>
</div>
{% endblock %}
```

OUTPUT:



Role of User Dashboard.html in Django:

- Serves as the main template for regular users to interact with the system.
- Displays a personalized dashboard showing user-specific content like upcoming events or welcome messages.
- Provides options for event interaction, such as viewing details or registering for events.
- Includes user-focused navigation, like menus for profile, events, settings, or logout.
- Renders dynamic content using Django's template tags to reflect real-time data from the backend.

Base.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>{% block title %}UNIVERSITY EVENTS{%
endblock %}</title>

    <style>
        /* General Styles */
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #121212;
            color: white;
            text-align: center;
        }
        header {
            background: #1e1e1e;
            padding: 20px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
            display: flex;
            justify-content: space-between;
            align-items: center;
            flex-wrap: wrap;
        }
        header h1 {
            margin: 0;
            font-size: 24px;
        }

        nav ul {
            list-style: none;
            padding: 0;
            margin: 0;
            display: flex;
            flex-wrap: wrap;
        }

        nav ul li {
            margin: 0 15px;
        }

        nav ul li a {
            text-decoration: none;
            color: #00ffcc;
            font-weight: bold;
            transition: color 0.3s ease-in-out;
        }
    </style>

```

```
nav ul li a:hover {
    color: #ffcc00;
}

.auth-links {
    display: flex;
    gap: 10px;
    align-items: center;
}

.auth-links a, .auth-links button {
    text-decoration: none;
    background: #00ffcc;
    color: black;
    padding: 8px 15px;
    border-radius: 5px;
    font-weight: bold;
    border: none;
    cursor: pointer;
    transition: background 0.3s;
}

.auth-links a:hover,
.auth-links button:hover {
    background: #ffcc00;
}

main {
    padding: 20px;
}

footer {
    background: #1e1e1e;
    padding: 10px;
    margin-top: 20px;
    font-size: 0.9em;
}

@media (max-width: 768px) {
    header {
        flex-direction: column;
        align-items: flex-start;
    }

    nav ul {
        flex-direction: column;
        align-items: flex-start;
        margin-top: 10px;
    }
}
```

```

        nav ul li {
            margin: 5px 0;
        }

        .auth-links {
            margin-top: 10px;
        }
    }
</style>
</head>
<body>
    <header>
        <h1>UNIVERSITY EVENTS</h1>

        <nav>
            <ul>
                <li><a href="{% url
'dashboard' %}">Dashboard</a></li>
                <li><a href="{% url 'events' %}">Events</a></li>
                <li><a href="{% url
'sponsors' %}">Sponsors</a></li>
                <li><a href="{% url
'contact' %}">Contact</a></li>
            </ul>
        </nav>

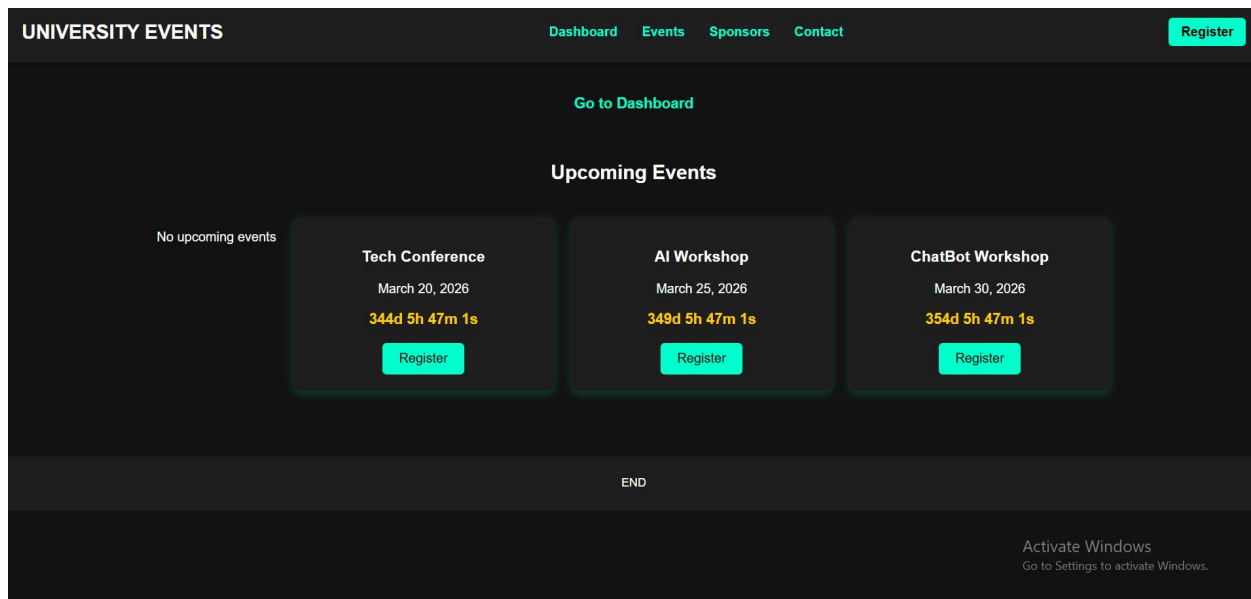
        <!-- Fixed Auth Links -->
        <div class="auth-links">
            {% if user.is_authenticated %}
                <form method="post" action="{% url 'logout' %}">
                    {% csrf_token %}
                    <button type="submit">Logout</button>
                </form>
            {% else %}
                <a href="{% url 'registration' %}">Register</a>
            {% endif %}
        </div>
    </header>

    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>END</p>
    </footer>
</body>
</html>

```

OUTPUT:



Role of base.html in Django:

- Acts as the **main layout template** for all other pages.
- Ensures a **consistent design** across the entire website.
- Provides a **shared header, navigation bar, and footer** for all templates.
- Simplifies maintenance by **avoiding code repetition**.
- Makes use of **Django's template inheritance** to allow other templates to insert content dynamically.
- Manages **user authentication display** (like showing logout if user is logged in).
- Improves responsiveness and mobile compatibility through shared CSS styles.
- Centralizes structure for easier **design updates** in one place.
- Keeps the project **organized and modular**.

Login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Login</title>
  <link
href="https://fonts.googleapis.com/css2?family=Orbitron:wght@4
00;600&display=swap" rel="stylesheet">
  <style>
    body {
      margin: 0; padding: 0;
      font-family: 'Orbitron', sans-serif;
      background: radial-gradient(circle at top, #0f0f1a,
#000);
      height: 100vh; display: flex;
      justify-content: center; align-items: center;
      color: #00f0ff;
    }
    .login-container {
      background: rgba(15,15,30,0.85);
      padding: 40px 30px;
      border-radius: 20px;
      box-shadow: 0 0 25px #00f0ff88;
      max-width: 420px; width: 100%;
    }
    h2 {
      text-align: center; margin-bottom: 30px;
      color: #00f0ff; text-shadow: 0 0 8px #00f0ff;
    }
    label, input { display: block; width: 100%; }

    label {
```

```

        margin-bottom: 6px; color: #00e0ff;
    }
    input[type="text"], input[type="password"] {
        padding: 12px; margin-bottom: 20px;
        border: 1px solid #00f0ff; border-radius: 8px;
        background: transparent; color: #fff;
        font-size: 14px; outline: none;
        box-shadow: 0 0 8px #00f0ff44;
    }
    input::placeholder { color: #aaa; }

    .button-container {
        display: flex; justify-content: space-between;
    }
    button {
        width: 48%; padding: 12px;
        border: none; border-radius: 8px;
        background: #00f0ff; color: #000;
        font-weight: bold; cursor: pointer;
        transition: 0.3s; box-shadow: 0 0 12px #00f0ff;
    }
    button:hover { background: #0ff; transform: scale(1.05); }

    p {
        color: #ff0066; text-align: center;
        font-weight: bold; text-shadow: 0 0 5px #ff0066;
    }
</style>
</head>
<body>
    <div class="login-container">
        <h2>Login</h2>

        {% if messages %}

```



```

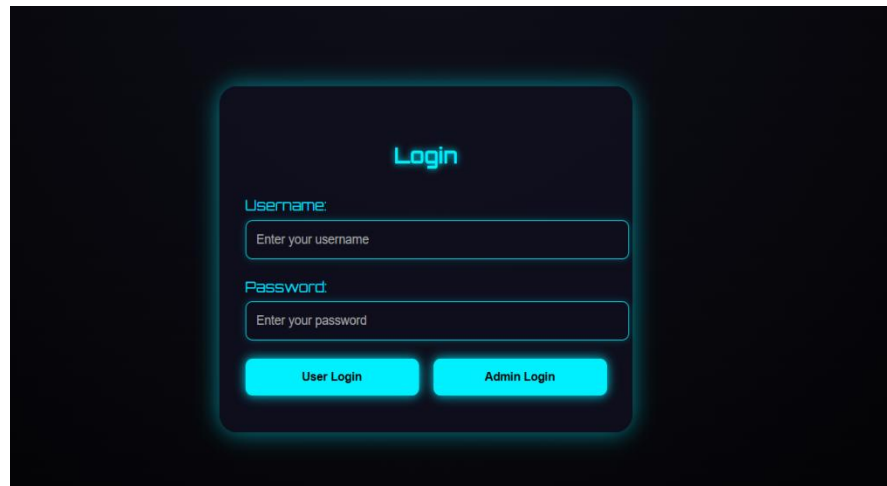
    {% for message in messages %}
        <p>{{ message }}</p>
    {% endfor %}
{% endif %}
<form method="POST">
    {% csrf_token %}
    <label>Username:</label>
    <input type="text" name="username" placeholder="Enter
your username" required>

    <label>Password:</label>
    <input type="password" name="password"
placeholder="Enter your password" required>

    <input type="hidden" name="login_type"
id="loginTypeInput">
<div class="button-container">
    <button type="submit"
onclick="setLoginType('user') ">User Login</button>
    <button type="submit"
onclick="setLoginType('admin') ">Admin Login</button>
</div>
</form>
</div>
<script>
    const setLoginType = type =>
document.getElementById('loginTypeInput').value = type;
</script>
</body>
</html>

```

OUTPUT:



Role of Login.html in Django:

Here are the key roles of login.html in your project in bullet points:

- **User Authentication Interface:**
 - Displays a form for entering username and password.
 - Allows both regular users and admins to log in.
 - Presents error messages when credentials are invalid.
- **Consistent Visual Branding:**
 - Styled with your futuristic neon theme (using colors like cyan and dark backgrounds).
 - Reinforces the overall aesthetic and branding of your project.
- **Integration with Django's Authentication System:**
 - Serves as the template rendered by your login view (either Django's built-in LoginView or a custom login_view).
 - Can be configured via URL names (e.g., /accounts/login/ or /login/).
- **Dynamic Context Handling:**
 - Can display context-sensitive messages (for example, a notification after successful registration).
 - Flexible to include additional elements based on user roles or states.
- **User Flow Gateway:**
 - Acts as the entry point for users after registration or when accessing protected content.
 - Upon successful authentication, redirects users to their appropriate dashboard (admin or user).

These points summarize the essential role and function of login.html in your project.


```

    }

    .hero {
        height: 100vh;
        display: flex; flex-direction: column; justify-content:
center; align-items: center;
        background:
url('https://source.unsplash.com/1600x900/?technology,future')
no-repeat center/cover;
    }

    .hero h1 {
        font-size: 3rem; text-transform: uppercase;
        text-shadow: 0 0 10px cyan;
    }

    .typing { font-size: 1.5rem; color: #0ff; }

    .hero button {
        margin-top: 20px; padding: 10px 20px;
        background: #0ff; border: none; color: #000;
        font-size: 18px; cursor: pointer; transition: 0.3s;
    }

    .hero button:hover { background: #fff; }

    .events {
        padding: 50px 20px;
    }

    .event-card {
        display: inline-block; background:
rgba(255,255,255,0.1);
        padding: 20px; margin: 20px; border-radius: 10px;
        box-shadow: 0 0 10px cyan; transition: 0.3s;
        width: 250px;
    }

    .event-card:hover { transform: scale(1.05); }

    .countdown { font-size: 1.2rem; color: #0ff; margin-top:
10px; }

    .live {

```

```

        color: red; font-weight: bold;
        animation: blink 1s infinite alternate;
    }
    @keyframes blink {
        from { opacity: 1; } to { opacity: 0.5; }
    }
    #about {
        display: flex; justify-content: center; align-items:
center;
        min-height: 100vh; text-align: center;
        padding: 50px 20px; background: rgba(0,0,0,0.8);
    }
    .about-container {
        max-width: 800px;
        background: rgba(255,255,255,0.1);
        padding: 30px; border-radius: 10px;
        box-shadow: 0 0 20px rgba(0,255,255,0.5);
    }
    h2 { color: cyan; }
    ul { list-style: none; padding: 0; }
    ul li { padding: 5px 0; }
    .footer {
        padding: 20px; background: black; text-align: center;
    }
</style>
</head>
<body>
<!-- Navigation Bar -->
<nav id="navbar">
    <ul>
        <li><a href="#home">Home</a></li>
        <li><a href="#about">About</a></li>
        <li><a href="#events">Events</a></li>
        <li><a href="#sponsors">Sponsors</a></li>

```

```

        <li><a href="#contact">Contact</a></li>
    </ul>
</nav>

<!-- Hero Section -->
<section class="hero" id="home">
    <h1>UNIVERSITY EVENTS</h1>
    <p class="typing">A National Event Symposium</p>
    <a href="/events/"><button>GET STARTED</button></a>
</section>

<!-- About Section -->
<section id="about">
    <div class="about-container">
        <h2>About the Event</h2>
        <p>Welcome to University Events... <strong>Join us and be
part of the journey!</strong></p>
        <h2>Who We Are</h2>
        <p>We host tech conferences, AI workshops, hackathons, and
networking sessions...</p>
        <h2>Our Mission</h2>
        <p>We aim to create a futuristic and knowledge-driven
environment...</p>
        <h2>What to Expect</h2>
        <ul>
            <li><strong>Keynotes:</strong> Industry leaders,
researchers, entrepreneurs</li>
            <li><strong>Workshops:</strong> AI, robotics,
blockchain, etc.</li>
            <li><strong>Competitions:</strong> Hackathons,
challenges</li>
            <li><strong>Networking:</strong> Panel discussions and
more</li>
            <li><strong>Career:</strong> Startup pitches,
recruiters</li>

```

```

</ul>

<h2>Why Attend?</h2>
<ul>
  <li><strong>Trends:</strong> Latest tech updates</li>
  <li><strong>Mentorship:</strong> Meet professionals</li>
  <li><strong>Collaboration:</strong> Team projects</li>
  <li><strong>Recognition:</strong> Prizes and
certificates</li>
</ul>

<h2>Vision</h2>
<p>We envision a world where students transform knowledge
into innovation...</p>
</div>
</section>

<!-- Events Section -->
<section id="events" class="events">
  <h2>Upcoming Events</h2>

  <div class="event-card">
    <h3>Tech Conference</h3>
    <p>March 20, 2026</p>
    <p class="countdown" id="countdown1"></p>
    <a href="{% url
'conference' %}"><button>Register</button></a>
  </div>

  <div class="event-card">
    <h3>AI Workshop</h3>
    <p>March 25, 2026</p>
    <p class="countdown" id="countdown2"></p>
    <a href="{% url

```

```

'ai_workshop' %}"><button>Register</button></a>
</div>

<div class="event-card">
  <h3>ChatBot Workshop</h3>
  <p>March 30, 2026</p>
  <p class="countdown" id="countdown3"></p>
  <a href="{% url
'chatbot_workshop' %}"><button>Register</button></a>
</div>
</section>

<!-- Sponsors Section -->
<section id="sponsors">
  <h2>Our Sponsors</h2>
  <p>Leading brands supporting innovation.</p>
</section>

<!-- Contact Section (imported from Django template) -->
{% include 'contact.html' %}

<!-- Footer -->
<div class="footer">
  <p>&copy; 2025 University Events. All rights reserved.</p>
</div>

<!-- Script for navbar and countdown timers -->
<script>
  // Shrink navbar on scroll
  window.addEventListener('scroll', () => {

document.getElementById("navbar").classList.toggle("shrink",
window.scrollY > 50);
  });

```



```

// Countdown logic
function updateCountdown(targetDate, elementId) {
    const eventDate = new Date(targetDate).getTime();
    const el = document.getElementById(elementId);
    if (!el) return;

    const interval = setInterval(() => {
        const now = new Date().getTime();
        const diff = eventDate - now;
        const d = Math.floor(diff / (1000 * 60 * 60 * 24));
        const h = Math.floor((diff % (1000 * 60 * 60 * 24)) /
(1000 * 60 * 60));
        const m = Math.floor((diff % (1000 * 60 * 60)) / (1000 *
60));
        const s = Math.floor((diff % (1000 * 60)) / 1000);

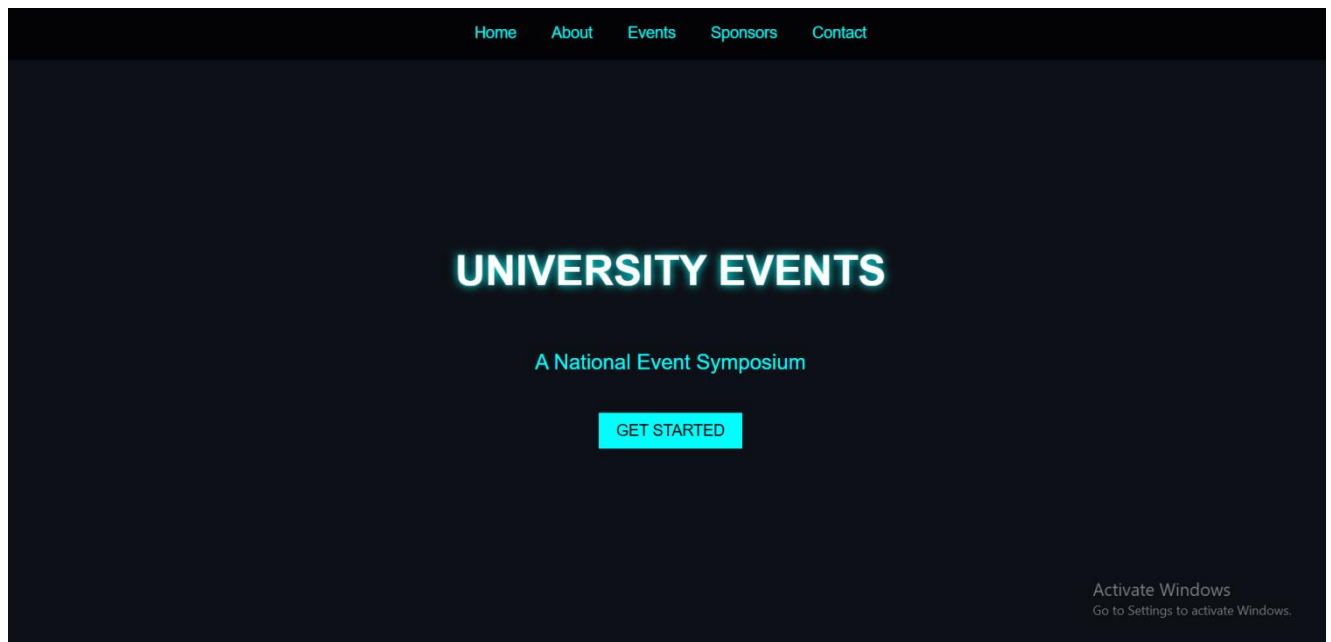
        el.innerHTML = `${d}d ${h}h ${m}m ${s}s`;
        if (diff < 0) {
            clearInterval(interval);
            el.innerHTML = '<span class="live">Event Live
Now</span>';
        }
    }, 1000);
}

// Initialize countdowns
updateCountdown("March 20, 2026 00:00:00", "countdown1");
updateCountdown("March 25, 2026 00:00:00", "countdown2");
updateCountdown("March 30, 2026 00:00:00", "countdown3");
</script>

</body>
</html>

```

OUTPUT:



Role of Home.html in Django:

Here are the key roles of **home.html** in your project in bullet points:

- **Main Landing Page:**
 - Acts as the first page visitors see.
 - Sets the overall tone and theme for the website.
- **Brand Presentation:**
 - Displays the University Events branding and logo.
 - Uses the neon, futuristic design to establish the project's look and feel.
- **Navigation Hub:**
 - Contains clear navigation links (Dashboard, Events, Sponsors, Contact, etc.).
 - Guides users to key sections of the site.
- **Hero Section & Call-to-Action:**
 - Features a hero area with a compelling title and tagline.
 - Includes a call-to-action (e.g., "Get Started") that directs users deeper into the site or event registration.
- **Introduction to Content:**
 - Provides a brief overview or introduction to what University Events is all about.
 - May highlight the benefits, mission, or upcoming events.
- **User Engagement:**
 - Serves as an entry point into the event ecosystem, inviting users to learn more, register, or log in.

These points summarize how **home.html** functions as a dynamic, welcoming landing page that sets the stage for exploring University Events and helps users navigate the site.

Events.html:

```
{% extends "base.html" %}

{% block title %}Upcoming Events{% endblock %}

{% block content %}
<a href="{% url 'dashboard' %}" class="dashboard-link">Go to
Dashboard</a>
<section class="events">
  <h2>Upcoming Events</h2>
  <div class="event-container">
    {% if events %}
      {% for event in events %}
        <div class="event-card">
          <h3>{{ event.title }}</h3>
          <p>{{ event.date }}</p>
          <p class="countdown"
id="countdown{{ forloop.counter }}"></p>
          <a href="{{ event.link }}">
            <button class="register-btn">Register</button>
          </a>
        </div>
      {% endfor %}
    {% else %}
      {% comment %} Static fallback {% endcomment %}
      {% with fallback_events=[
        {'title': 'Tech Conference', 'date': 'March 20, 2026',
'url_name': 'conference'},
        {'title': 'AI Workshop', 'date': 'March 25, 2026',
'url_name': 'ai_workshop'},
        {'title': 'ChatBot Workshop', 'date': 'March 30,
2026', 'url_name': 'chatbot_workshop'}
      ] %}
        {% for item in fallback_events %}
```

```

        <div class="event-card">
            <h3>{{ item.title }}</h3>
            <p>{{ item.date }}</p>
            <p class="countdown"
id="countdown{{ forloop.counter }}"></p>
            <a href="{% url item.url_name %}">
                <button class="register-btn">Register</button>
            </a>
        </div>
    {% endfor %}
{% endwith %}
{% endif %}
</div>
</section>
<style>
    .dashboard-link {
        display: block;
        text-align: center;
        font-size: 18px;
        margin: 20px 0;
        color: #00ffcc;
        text-decoration: none;
        font-weight: bold;
    }
    .dashboard-link:hover { color: #ffcc00; }
    .events { text-align: center; padding: 20px; }
    .event-container {
        display: flex;
        justify-content: center;
        gap: 20px;
        flex-wrap: wrap;
        padding: 20px;
    }

```

```

.event-card {
  background: #1e1e1e;
  padding: 20px;
  width: 280px;
  border-radius: 10px;
  box-shadow: 0 4px 10px rgba(0, 255, 204, 0.2);
  transition: transform 0.3s ease-in-out;
  text-align: center;
}

.event-card:hover { transform: scale(1.05); }

.register-btn {
  background: #00ffcc;
  color: black;
  border: none;
  padding: 10px 20px;
  font-size: 16px;
  border-radius: 5px;
  cursor: pointer;
  transition: 0.3s;
}

.register-btn:hover { background: #ffcc00; }

.countdown {
  font-size: 18px;
  font-weight: bold;
  color: #ffcc00;
}

@media (max-width: 768px) {
  .event-container { flex-direction: column; align-items:
center; }
}
</style>
<script>
const countdowns = [
  { id: "countdown1", date: "March 20, 2026 00:00:00" },

```

```

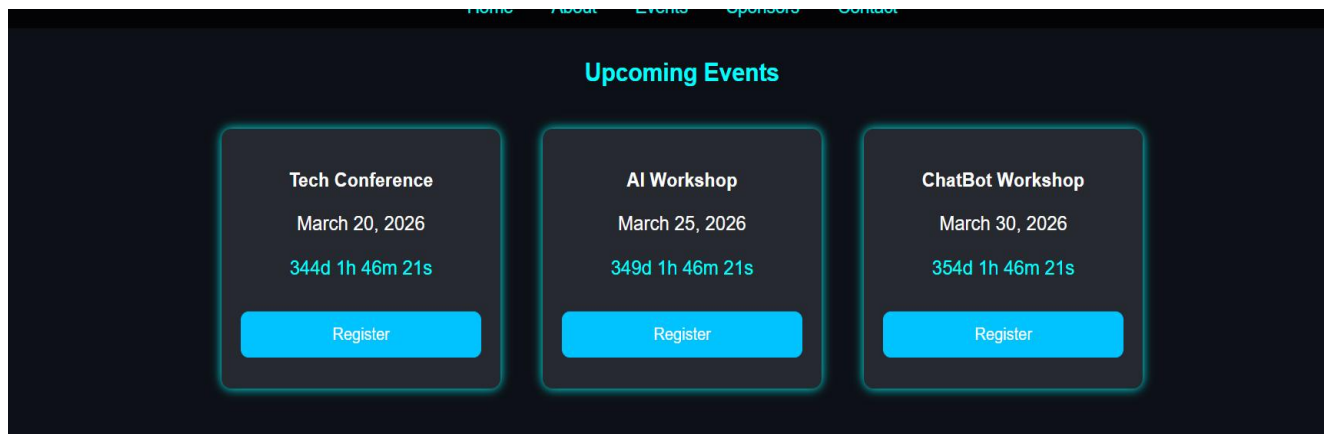
    { id: "countdown2", date: "March 25, 2026 00:00:00" },
    { id: "countdown3", date: "March 30, 2026 00:00:00" },
  ];

  function startCountdown(targetDate, elementId) {
    const countDownDate = new Date(targetDate).getTime();
    const el = document.getElementById(elementId);
    if (!el) return;
    const interval = setInterval(() => {
      const now = Date.now();
      const dist = countDownDate - now;
      if (dist <= 0) {
        clearInterval(interval);
        el.textContent = "Event Live Now!";
        return;
      }
      const d = Math.floor(dist / (1000 * 60 * 60 * 24));
      const h = Math.floor((dist / (1000 * 60 * 60)) % 24);
      const m = Math.floor((dist / (1000 * 60)) % 60);
      const s = Math.floor((dist / 1000) % 60);
      el.textContent = `${d}d ${h}h ${m}m ${s}s`;
    }, 1000);
  }

  countdowns.forEach(c => startCountdown(c.date, c.id));
</script>
{% endblock %}

```

OUTPUT:



Role of Events.html in Django:

Here are the key roles of **events.html** in your project summarized in bullet points:

- **Listing All Events:**
 - Displays a collection of upcoming (and possibly past) events.
 - Dynamically renders event cards or items, each with details such as title, date, and registration link.
- **User Navigation:**
 - Serves as a central hub for users to browse different events.
 - Provides links or buttons that allow users to quickly access event-specific pages (e.g., Tech Conference, AI Workshop, ChatBot Workshop).
- **Visual Engagement:**
 - Utilizes your neon, futuristic theme to present events in an attractive, consistent layout.
 - Uses hover effects and countdown timers on event cards to entice user interaction and convey event urgency.
- **Call-to-Action:**
 - Includes direct actions such as “Register” buttons to let users quickly sign up for events.
 - Encourages user engagement by providing clear pathways to event registration and details.
- **Dynamic Content Integration:**
 - Pulls event information from the database, ensuring that users see updated and relevant events.
 - Potentially supports filtering or sorting of events if extended in the future.

These points reflect how **events.html** functions as a crucial part of your project by listing and presenting the event offerings in a visually appealing, user-friendly manner while aligning with your futuristic neon design.

Logout.html:

```
{% extends "base.html" %}

{% block content %}
<div class="logout-container">
    <h2 class="glow-text">You have been logged out</h2>
    <p class="glow-sub">Redirecting you to the dashboard...</p>
    <p class="glow-sub">
        <a href="{% url 'dashboard' %}" class="redirect-link">click here</a>.
    </p>
</div>

<script>
    setTimeout(() => {
        window.location.href = "{% url 'dashboard' %}";
    }, 2000); // Redirect after 2 seconds
</script>

<style>
.logout-container {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    height: 70vh;
    text-align: center;
    background: rgba(0, 0, 0, 0.6);
    padding: 40px;
    border-radius: 15px;
    margin: 60px auto;
    width: 60%;
}
```



```
    box-shadow: 0 0 20px rgba(0, 255, 255, 0.4);
}

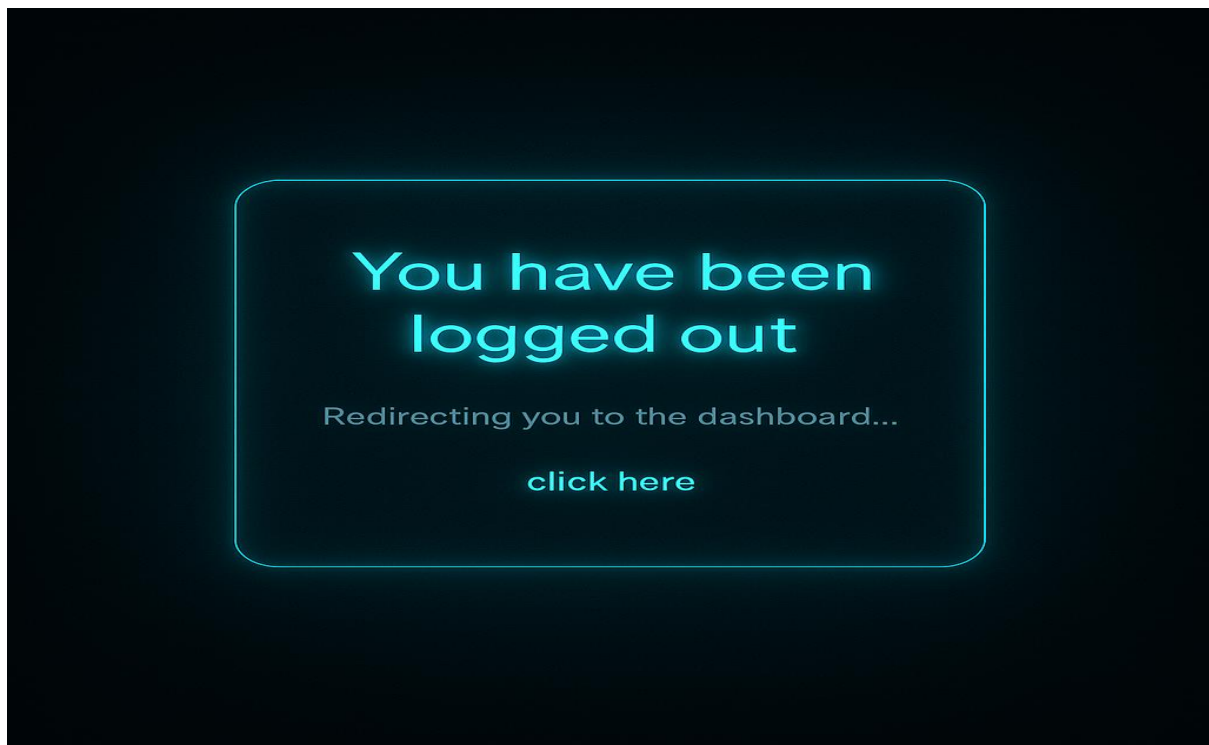
.glow-text {
    color: #0ff;
    font-size: 2rem;
    text-shadow: 0 0 10px #0ff, 0 0 20px #0ff;
}

.glow-sub {
    color: #ccc;
    margin-top: 10px;
}

.redirect-link {
    color: #0ff;
    font-weight: bold;
    text-decoration: none;
    border-bottom: 1px solid #0ff;
}

.redirect-link:hover {
    text-shadow: 0 0 5px #0ff;
}
</style>
{% endblock %}
```

OUTPUT:



Role of Logout.html in Django:

- **Displays logout confirmation** – Notifies the user that they have been successfully logged out of the system.
- **Enhances user experience** – Provides a clear and stylish message instead of redirecting immediately without feedback.
- **Uses automatic redirection** – Includes JavaScript that redirects users to the dashboard after a short delay.
- **Acts as a feedback mechanism** – Confirms that the logout action was completed, which builds user trust.
- **Maintains design consistency** – Inherits styles and structure from a shared `base.html` layout, keeping the UI uniform.
- **Uses visual effects** – Features glowing text, soft background, and animations to match a futuristic or tech-themed UI.
- **Offers manual navigation** – Provides a clickable link in case the auto-redirect fails or the user wants to move faster.

Registration.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Register for {{ event_name }}</title>
  <style>
    body {
      margin: 0;
      background: #0d0d0d;
      color: white;
      font-family: Arial, sans-serif;
      text-align: center;
    }

    .container {
      max-width: 450px;
      margin: 50px auto;
      padding: 30px;
      background: rgba(255,255,255,0.1);
      border-radius: 12px;
      box-shadow: 0 0 15px rgba(0,255,255,0.3);
    }

    h1 { color: cyan; }

    form { display: flex; flex-direction: column; text-align:
left; }

    label {
      font-weight: bold;
      margin-top: 12px;
    }

    input {
      width: 100%;
      padding: 14px;
      margin-top: 6px;
      font-size: 16px;
      border: none;
      border-radius: 8px;
      background: rgba(255,255,255,0.2);
      color: white;
    }

    input:focus {
      outline: none;
      border: 2px solid cyan;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Register</h1>
    <form>
      <label>Name</label>
      <input type="text">
      <label>Email</label>
      <input type="text">
      <label>Password</label>
      <input type="password">
      <label>Confirm Password</label>
      <input type="password">
      <input type="submit" value="Register">
    </form>
  </div>
</body>
</html>
```

```
    background: rgba(255,255,255,0.3);
}

.input-container {
    position: relative;
}

.toggle-password {
    position: absolute;
    right: 10px;
    top: 50%;
    transform: translateY(-50%);
    cursor: pointer;
    color: cyan;
}

.error {
    color: red;
    font-size: 14px;
    margin-top: 5px;
}

button {
    background: cyan;
    color: black;
    font-weight: bold;
    padding: 16px;
    font-size: 18px;
    margin-top: 20px;
    border: none;
    border-radius: 8px;
    cursor: pointer;
}

button:hover { background: white; }

.back-links {
    margin-top: 25px;
}

.back-links a {
    color: cyan;
    font-weight: bold;
    text-decoration: none;
}

.back-links a:hover {
    color: white;
    text-decoration: underline;
}
```

```

.modal {
  display: none;
  position: fixed;
  inset: 0;
  background: rgba(0,0,0,0.8);
  justify-content: center;
  align-items: center;
  z-index: 1000;
}

.modal-content {
  background: #111;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 0 20px cyan;
  text-align: center;
  max-width: 400px;
}

.modal-content h2 { color: cyan; margin-bottom: 20px; }

.modal-content button {
  background: cyan;
  padding: 12px 25px;
  border: none;
  border-radius: 8px;
  font-weight: bold;
  font-size: 16px;
  cursor: pointer;
}

.modal-content button:hover { background: white; }
</style>

<script>
  const togglePasswordVisibility = (id, iconId) => {
    const input = document.getElementById(id);
    const icon = document.getElementById(iconId);
    const show = input.type === "password";
    input.type = show ? "text" : "password";
    icon.innerText = show ? " " : " ";
  };

  const getCSRFToken = () =>
document.querySelector('[name=csrfmiddlewaretoken]').value;

  const goToLogin = () => window.location.href = "{% url
'login' %}";

  window.onload = () => {
    const form =

```

```

document.getElementById("registrationForm");
    form.onsubmit = (e) => {
        e.preventDefault();
        const errorText = document.getElementById("error-
message");
        errorText.innerText = "";

        const formData = new FormData(form);
        if (formData.get("password") !==
formData.get("confirm_password")) {
            errorText.innerText = "Passwords do not match!";
            return;
        }

        fetch("{% url 'submit_registration' %}", {
            method: "POST",
            headers: { "X-CSRFToken": getCSRFToken() },
            body: formData
        })
        .then(res => res.json())
        .then(data => {
            if (data.success) {
                form.reset();

document.getElementById("successModal").style.display =
"flex";
                setTimeout(goToLogin, 2000);
            } else {
                errorText.innerText = data.error || "Registration
failed!";
            }
        })
        .catch(() => errorText.innerText = "Something went
wrong.");
    };
};
</script>
</head>
<body>

<div class="container">
    <h1>Register for {{ event_name }}</h1>
    <form id="registrationForm" method="post">
        {% csrf_token %}
        <input type="hidden" name="event_name"
value="{{ event_name }}">

        <label>Full Name:</label>
        <input type="text" name="name" required>

        <label>Email:</label>

```

```

<input type="email" name="email" required>

<label>Phone Number:</label>
<input type="tel" name="phone" required>

<label>Password:</label>
<div class="input-container">
    <input type="password" id="password" name="password"
required>
    <span class="toggle-password" id="togglePassword"
onclick="togglePasswordVisibility('password',
'togglePassword')"> </span>
</div>

<label>Confirm Password:</label>
<div class="input-container">
    <input type="password" id="confirm_password"
name="confirm_password" required>
    <span class="toggle-password" id="toggleConfirmPassword"
onclick="togglePasswordVisibility('confirm_password',
'toggleConfirmPassword')"> </span>
</div>

<p id="error-message" class="error"></p>
<button type="submit">Submit Registration</button>
</form>

<div class="back-links">
    <a href="{% url 'login' %}">Have an account? Login...</a>
</div>
</div>

<div id="successModal" class="modal">
    <div class="modal-content">
        <h2>Registration Successful </h2>
        <p>You can now log in to your account.</p>
        <button onclick="goToLogin()">Go to Login</button>
    </div>
</div>

</body>
</html>

```

OUTPUT:

The image shows a registration form titled 'Register for' in a neon cyan font. The form is set against a dark background. It contains five input fields: 'Full Name:', 'Email:', 'Phone Number:', 'Password:', and 'Confirm Password:'. Each field has a corresponding input box with a small eye icon to the right, likely for toggling password visibility. Below the input fields is a large, bright cyan button labeled 'Submit Registration'. At the very bottom, there is a link that says 'Have an account? Login...'.

Role of Registration.html in Django:

Here are the key roles of **registration.html** in your project, summarized in bullet points:

- **User Registration Interface:**
 - Provides a clean and themed form for new users to register for a specific event.
 - Collects essential information like full name, email, phone number, and password details.
- **Client-Side Validation:**
 - Uses JavaScript to validate critical fields (e.g., ensuring password and confirm password match) before submission.
 - Displays error messages dynamically to guide users in correcting issues immediately.
- **AJAX Integration for Smooth Experience:**
 - Submits registration details via AJAX to avoid full-page reloads.
 - Displays a success modal (popup) dynamically when registration is successful, enhancing user experience.
- **Consistent Neon Futuristic Theme:**
 - Styled with neon colors (cyan, dark backgrounds, and glowing effects) to match the overall aesthetic of the University Events project.
 - Reinforces brand consistency and creates a modern, visually engaging environment.
- **Redirection and Navigation Aid:**
 - Provides a link or auto-redirect option that guides successfully registered users to the login page.
 - Includes back-links to easily navigate to the login page if the user already has an account.

Contact.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Contact Us</title>
<style>
  body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #0d1117;
    color: #fff;
  }
  #contact {
    padding: 60px 20px;
    text-align: center;
  }
  h2, h3 {
    color: #00c3ff;
    margin-bottom: 15px;
  }
  .contact-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px,
1fr));
    gap: 30px;
    max-width: 1000px;
    margin: auto;
  }
  .box {
```

```
background: #161b22;
padding: 20px;
border-radius: 12px;
box-shadow: 0 4px 10px rgba(0, 195, 255, 0.2);
text-align: left;
}
form input,
form textarea,
form button {
width: 100%;
padding: 12px;
margin: 10px 0;
font-size: 1rem;
border-radius: 8px;
border: 1px solid #00c3ff;
background: #0d1117;
color: white;
outline: none;
}
form textarea {
height: 120px;
resize: none;
}
form button {
background: #00c3ff;
border: none;
font-size: 1.2rem;
cursor: pointer;
transition: 0.3s;
}
form button:hover {
background: #0077b6;
```

```

        transform: scale(1.05);
    }
    .contact-map iframe {
        width: 100%;
        height: 300px;
        border: none;
        border-radius: 12px;
    }
    .contact-details p {
        margin: 8px 0;
        font-size: 1.05rem;
    }
    @media (max-width: 768px) {
        .contact-grid {
            grid-template-columns: 1fr;
        }
    }
</style>
</head>
<body>
<section id="contact">
    <h2>Reach out to Us</h2>
    <div class="contact-grid">
        <!-- Contact Form -->
        <div class="box">
            <h3>Contact Form</h3>
            <form id="contactForm">
                {% csrf_token %}
                <input type="text" name="name" placeholder="Your Name"
required>
                <input type="email" name="email" placeholder="Your
Email" required>
                <input type="tel" name="mobile" placeholder="Your

```

```

Mobile Number" required pattern="[0-9]{10}">

    <textarea name="message" placeholder="Message"
required></textarea>

    <button type="submit">Send Message</button>

</form>
</div>

<!-- Google Map -->
<div class="box contact-map">
    <h3>Our Location</h3>
    <iframe

src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d391
8.984276336993!2d-
74.00601568538462!3d40.71277614828256!2m3!1f0!2f0!3f0!3m2!1i10
24!2i768!4f13.1!3m3!1m2!1s0x89c25a1784b9e5d9%3A0x4f4a535e0e3df
18!2sJNTU-
GV%20COLLEGE%20OF%20ENGINEERING!5e0!3m2!1sen!2sus!4v1617280257
934!5m2!1sen!2sus"

        loading="lazy" allowfullscreen>
    </iframe>
</div>

<!-- Contact Info -->
<div class="box contact-details">
    <h3>Contact Information</h3>
    <p><strong> Address:</strong> Vizianagaram, Andhra
Pradesh - 535003</p>
    <p><strong> Email:</strong> info@example.com</p>
    <p><strong> Phone:</strong> xxxxxxxxxxx</p>
</div>
</div>
</section>

<script>

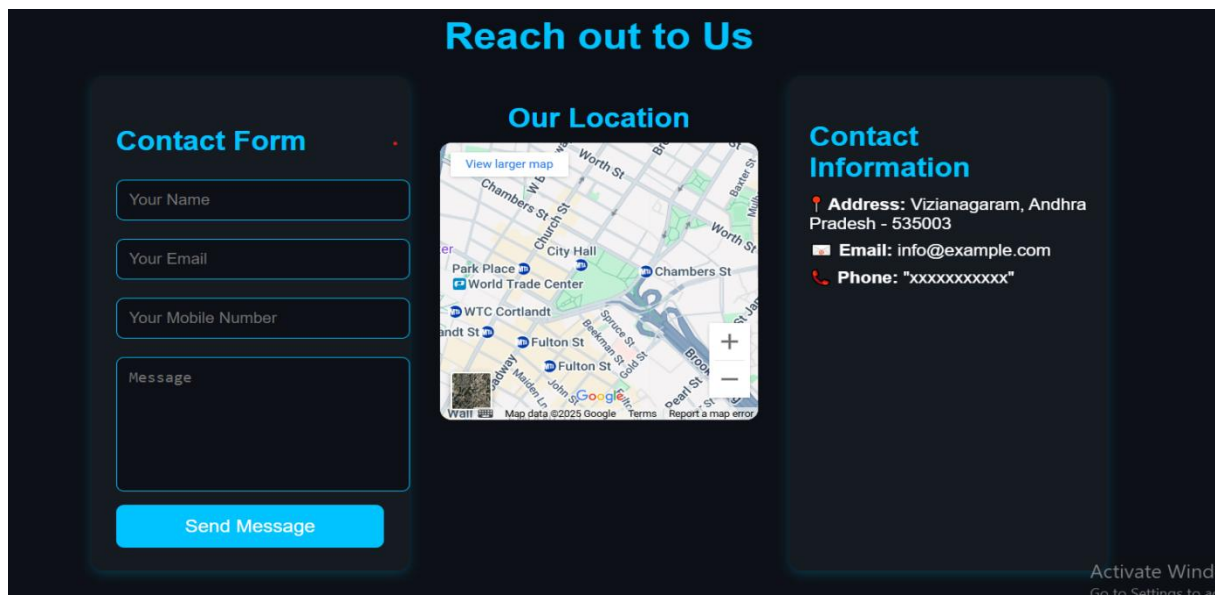
```

```
document.getElementById("contactForm").onsubmit = function(e)
{
    e.preventDefault();
    const formData = new FormData(this);

    fetch("{% url 'contact' %}", {
        method: "POST",
        body: formData,
        headers: {
            "X-Requested-With": "XMLHttpRequest",
            "X-CSRFToken":
document.querySelector("[name=csrfmiddlewaretoken]").value
        }
    })
    .then(res => res.json())
    .then(data => alert(data.message))
    .catch(err => console.error("Error:", err));
};
</script>

</body>
</html>
```

OUTPUT:



Role of Contact.html in Django:

- **User Communication Portal**

Allows users to send messages, queries, or feedback to event organizers.

- **Supports Multiple Purposes**

Visitors can contact for:

- Event-related questions
- Sponsorship offers
- Volunteering opportunities
- Technical issues (e.g., login problems)

- **Displays Organizer Info**

Shows contact details like:

- Email address
- Phone number
- Office/location (optional)

- **Includes a Contact Form**

Typically has fields like:

- Name
- Email
- Subject (optional dropdown)
- Message

Conference.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Tech Conference 2025</title>
  <link rel="stylesheet" href="styles.css"> <!-- Link to
your CSS file -->
  <style>
    body {
      background: #0d1117;
      color: #ffffff;
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;
    }

    .container {
      max-width: 900px;
      margin: 50px auto;
      padding: 20px;
      background: #161b22;
      border-radius: 12px;
      box-shadow: 0 4px 10px rgba(0, 195, 255, 0.2);
      text-align: center;
    }

    h1 {
      color: #00c3ff;
    }

    p {
      font-size: 1.2rem;
      margin-bottom: 20px;
    }

    .register-form {
      text-align: left;
      max-width: 400px;
      margin: auto;
    }

    input, button {
      width: 100%;
      padding: 12px;
      margin: 10px 0;
      border: 1px solid #00c3ff;
      border-radius: 8px;
```

```

        background: #0d1117;
        color: #ffffff;
        font-size: 1rem;
        outline: none;
    }

    button {
        background: #00c3ff;
        cursor: pointer;
        transition: 0.3s;
    }

    button:hover {
        background: #0077b6;
        transform: scale(1.05);
    }
</style>
</head>
<body>

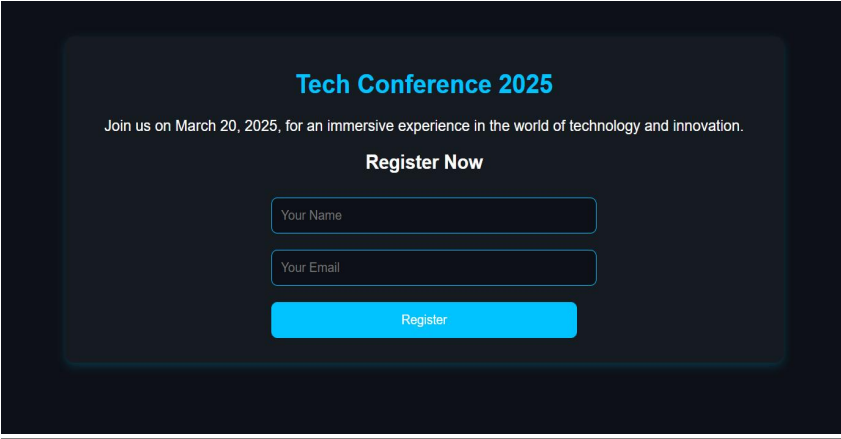
    <div class="container">
        <h1>Chatbot Development 2025</h1>
        <p>Join us on March 30, 2025, for an immersive
experience in the world of technology and innovation.</p>

        <h2>Register Now</h2>
        <form class="register-form">
            <input type="text" name="name" placeholder="Your
Name" required>
            <input type="email" name="email" placeholder="Your
Email" required>
            <button type="submit">Register</button>
        </form>
    </div>

</body>
</html>

```


OUTPUT:

The image shows a registration form for 'Tech Conference 2025'. The form is centered on a dark background with glowing blue borders. At the top, the title 'Tech Conference 2025' is displayed in a glowing blue font. Below the title, a line of text reads: 'Join us on March 20, 2025, for an immersive experience in the world of technology and innovation.' Underneath this, the text 'Register Now' is shown. The form contains two input fields: 'Your Name' and 'Your Email', both with glowing blue borders. At the bottom of the form is a glowing blue button labeled 'Register'.

Role of conference.html in Django:

- **Dedicated Page for Conference Events**

Specifically showcases university conferences (academic, tech, research-based events).

- **Detailed Conference Information**

Displays key details like:

- Conference title & theme
- Date & time
- Venue/location
- Speaker/panel list
- Registration info or links
- Agenda or schedule (optional)

- **Countdown Timer for Conference Start**

Includes a **live countdown** just like other event cards on the homepage.

- **"Live Now" Status Integration**

Shows a glowing "LIVE" tag when the conference is in progress (real-time status update).

- **Stylized with Neon Theme**

Consistent futuristic design: glowing borders, animated timers, dark background.

- **Call-to-Action (CTA)**

May include buttons like:

- "Register Now"
- "Join Live Stream"
- "View Schedule"

Chatbot.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Tech Conference 2025</title>
  <link rel="stylesheet" href="styles.css"> <!-- Link to
your CSS file -->
  <style>
    body {
      background: #0d1117;
      color: #ffffff;
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;
    }

    .container {
      max-width: 900px;
      margin: 50px auto;
      padding: 20px;
      background: #161b22;
      border-radius: 12px;
      box-shadow: 0 4px 10px rgba(0, 195, 255, 0.2);
      text-align: center;
    }

    h1 {
      color: #00c3ff;
    }

    p {
      font-size: 1.2rem;
      margin-bottom: 20px;
    }

    .register-form {
      text-align: left;
      max-width: 400px;
      margin: auto;
    }

    input, button {
      width: 100%;
      padding: 12px;
      margin: 10px 0;
      border: 1px solid #00c3ff;
      border-radius: 8px;
```

```

        background: #0d1117;
        color: #ffffff;
        font-size: 1rem;
        outline: none;
    }

    button {
        background: #00c3ff;
        cursor: pointer;
        transition: 0.3s;
    }

    button:hover {
        background: #0077b6;
        transform: scale(1.05);
    }
</style>
</head>
<body>

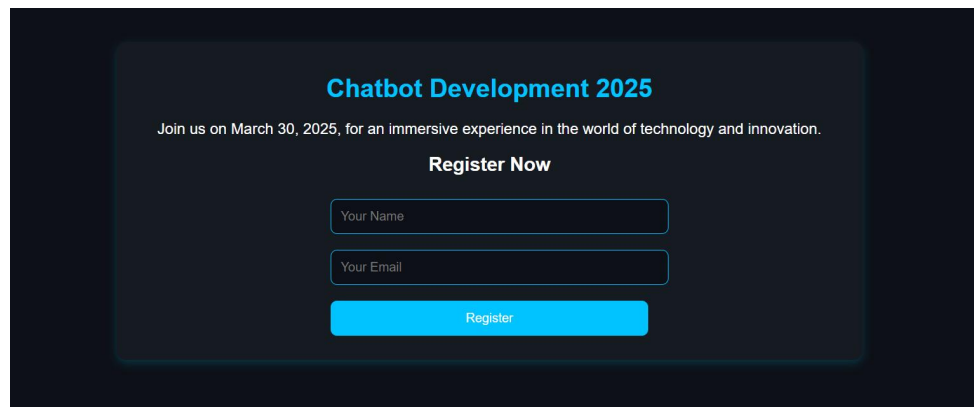
    <div class="container">
        <h1>Chatbot Development 2025</h1>
        <p>Join us on March 30, 2025, for an immersive
experience in the world of technology and innovation.</p>

        <h2>Register Now</h2>
        <form class="register-form">
            <input type="text" name="name" placeholder="Your
Name" required>
            <input type="email" name="email" placeholder="Your
Email" required>
            <button type="submit">Register</button>
        </form>
    </div>

</body>
</html>

```

OUTPUT



Role of Chatbot.html in Django:

- **Interactive Help Assistant Page**

Acts as a smart assistant to help users navigate your website or get quick answers.

- **User-Friendly Support Tool**

Provides real-time responses to user queries like:

- “When is the next event?”
- “How to register?”
- “Who are the sponsors?”
- “Where is the venue?”

- **24/7 Instant Assistance**

Available anytime for users to get help without waiting for admin responses.

- **Improves User Engagement**

Keeps visitors engaged and makes your site feel more interactive and modern.

- **Consistent with Neon Theme**

Styled with your futuristic look: glowing borders, dark background, animated elements.

- **Useful for Common FAQs**

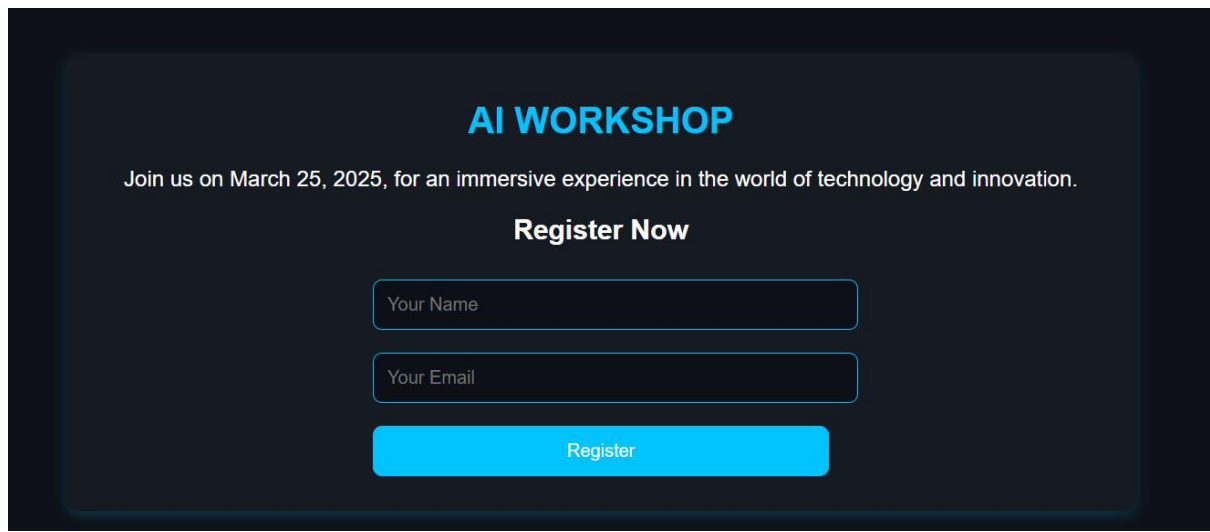
Can be programmed to answer frequently asked questions about:

- Events
- Registration
- Login issues
- Contact info

Ai.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
<title>Tech Conference 2025</title>
<style>
body {
margin: 0;
font-family: Arial, sans-serif;
background: #0d1117;
color: #fff;
}
.container {
max-width: 900px;
margin: 50px auto;
padding: 20px;
text-align: center;
background: #161b22;
border-radius: 12px;
box-shadow: 0 4px 10px rgba(0, 195, 255, 0.2);
}
h1 { color: #00c3ff; }
p { font-size: 1.2rem; }
form {
max-width: 400px;
margin: 20px auto 0;
text-align: left;
}
input, button {
```

```
width: 100%;
padding: 12px;
margin: 10px 0;
border-radius: 8px;
font-size: 1rem;
border: 1px solid #00c3ff;
background: #0d1117;
color: #fff;
}
button {
background: #00c3ff;
cursor: pointer;
transition: 0.3s;}
button:hover {
background: #0077b6;
transform: scale(1.05);}
</style>
</head>
<body>
<div class="container">
<h1>AI WORKSHOP</h1>
<p>Join us on March 25, 2025, for an immersive experience in
the world of technology and innovation.</p>
<h2>Register Now</h2>
<form>
<input type="text" placeholder="Your Name" required>
<input type="email" placeholder="Your Email" required>
<button type="submit">Register</button>
</form>
</div>
</body>
</html>
```

OUTPUT:


The image shows a registration form for an 'AI WORKSHOP'. The title 'AI WORKSHOP' is in large, bold, blue letters. Below it, a line of text says 'Join us on March 25, 2025, for an immersive experience in the world of technology and innovation.' Underneath this is a 'Register Now' button in white text on a dark blue background. Below the button are two input fields: 'Your Name' and 'Your Email', both with light blue borders and placeholder text. At the bottom is a large, solid blue 'Register' button.

Role of Ai.html in Django:

- **Dedicated template for AI-related content** – Used to display information about AI events, workshops, courses, or features in the system.
- **Serves as a frontend interface** for users to interact with AI-focused activities (e.g., viewing event details or registering for workshops).
- **Connected to Django views** that handle data processing and render the `ai.html` file when users access AI-related URLs.
- **Displays dynamic data** using Django template tags (`{{ }}` and `{% %}`), like workshop dates, registration status, or AI topics.
- **Provides a clean, themed layout** for AI events (often with tech-style design, neon colors, etc., to match the AI theme).
- **Used in role-based templates** to show different options depending on whether the user is a student, teacher, or admin.
- **Can include forms** for AI event registration, feedback, or interest submission directly on the page.

Database:

A database is an organized collection of data that is stored and managed in a way that makes it easy to retrieve, update, and manage information. Databases are used to store data in a structured format, allowing efficient access, management, and modification.

Step 1: Check if SQLite3 is Already Installed

```
sqlite3 --version
```

Step 2: Install SQLite3 (if not already installed)

On Windows:

1. Download the SQLite3 command-line tool from the official website:
SQLite Downloads
2. Download the "Precompiled Binaries for Windows" (usually a ZIP file).
3. Extract the ZIP file and place sqlite3.exe in a folder (e.g., C:\sqlite).
4. Add the folder to your system's PATH environment variable:
 - Search for "Environment Variables" in the Start menu.
 - Click on "Environment Variables."
 - In "System variables," select "Path" and click "Edit."
 - Add the folder path (e.g., C:\sqlite) and click OK.

```
sqlite3 --version
```

Step 4: Using SQLite3 with Django

1. Since Django uses SQLite3 as the default database, you don't need to install any additional drivers. Just make sure your settings.py file has the following configuration


```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

- Step 5: Run Migrations to Create the Database
- After setting up, run:
- `python manage.py migrate`
- Step 1: Open the Django DB Shell
- Make sure your virtual environment is activated, then use the following command:
- `python manage.py db shell`
- Step 2: Common SQLite Commands
- Once you're inside the SQLite shell, you can use the following commands:
- List All Tables:
- `.tables`
- Step3: View Table Schema:
- `.schema table_name;`

Step4: Show All Data in a Table

- `SELECT * FROM table_name;`
- Exit the SQLite Shell:

`.exit`



What is forms.py in Django:

In Django, forms.py is a dedicated Python file used to define and manage forms for collecting user input. It simplifies how you create HTML forms, handle validation, and process submitted data — all in a secure and efficient way.

Why Use forms.py in Django?

Using forms.py in Django brings a lot of benefits when dealing with user input:

- **Makes form creation easier**
You define forms using Python classes instead of writing raw HTML for every field.
- **Built-in validation**
Django automatically checks if the input is valid (e.g., correct data types, required fields filled, etc.).
- **Works seamlessly with models**
With `ModelForm`, forms are generated directly from your models, making CRUD operations faster and cleaner.
- **Improves security**
Protects your app from threats like **SQL Injection** and **CSRF attacks** by handling these issues under the hood.

Types of Forms in Django:

- Django Forms (`forms.Form`) – Used for manually creating forms
- **Model Forms (`forms.ModelForm`)** – Used to create forms directly from a Django model

```
from django import forms
from .models import Contact # Ensure Contact model exists
from .models import Event

class ContactForm(forms.ModelForm):
    class Meta:
        model = Contact
        fields = ['name', 'email', 'message'] # Ensure these fields exist in
        Contact model

class EventRegistrationForm(forms.ModelForm):
    class Meta:
        model = Event # Ensure Event model is correctly imported
        fields = ['name', 'email', 'phone_number'] # Adjust as needed
```

What is models.py in Django:

In Django, models.py is where you define the database structure using Python code. Django models act as a bridge between the database and the application, allowing you to create, read, update, and delete records easily.

```
from django.db import models

class Event(models.Model):
    name = models.CharField(max_length=255)
    email = models.EmailField() # Ensure this field
exists
    phone_number = models.CharField(max_length=15)
# Ensure this field exists

    def __str__(self):
        return self.name

class EventRegistration(models.Model):
    event = models.ForeignKey(Event,
on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    email = models.EmailField()

class Contact(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    message = models.TextField()

from django.db import models

class Item(models.Model):
    name = models.CharField(max_length=255)
    price = models.DecimalField(max_digits=10,
decimal_places=2)

    def __str__(self):
        return self.name

from django.db import models

class Order(models.Model):
    order_id = models.CharField(max_length=100,
unique=True)
    item = models.ForeignKey('Item',
on_delete=models.CASCADE) # Assuming Item exists
```

```

        quantity = models.PositiveIntegerField()
        created_at =
models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.order_id

from django.contrib.auth.models import AbstractUser
from django.db import models

class CustomUser(AbstractUser):
    email = models.EmailField(unique=True)  # ✓ Make
email unique

    # Optional: add any custom fields like phone,
user_type, etc.
    phone = models.CharField(max_length=15,
blank=True)
    user_type = models.CharField(max_length=10,
default='user')

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['username'] # or include
first_name, last_name, etc. as needed

    def __str__(self):
        return self.email

```

Run Migrations to Create Database Tables:

After defining your models, run the following commands to apply them to the database:

- `python manage.py makemigrations`
- `python manage.py migrate`

Register Models in `admin.py`:

Admin.py:

```
from django.contrib
import admin
from .models
import Room,
Booking
class RoomAdmin(admin.ModelAdmin):
    list_display = ('name', 'capacity', 'teacher', 'is_active') # Display relevant fields in
    the list search_fields = ('name', 'teacher__username') # Allow search by
    room name and teacher's
    username
    list_filter = ('is_active', 'teacher') # Add filters for active status and teacher

class BookingAdmin(admin.ModelAdmin):
    list_display = ('room', 'user', 'date', 'booked_by') # Display room, user, date, and
    who booked the room
    list_filter = ('room', 'user', 'date') # Add filters for room, user, and date
    search_fields = ('room__name', 'user__username') # Allow search by room name
    and user

# Register the models with custom admin classes
admin.site.register(Room, RoomAdmin)
admin.site.register(Booking, BookingAdmin)
```

Deploying Django Web Application on Cloud:

What is Deployment?

Deployment is the process of **making your Django application live and accessible on the internet**, typically by hosting it on a **cloud server or platform**. This allows users around the world to interact with your app in real time.

In short, deployment moves your Django project from local development to a **production-ready environment**.

Features & Benefits of Deployment:

- **Global Access:** Makes your website accessible online 24/7.
- **Secure Environment:** Uses production-ready settings with security features like HTTPS, firewalls, and authentication.
- **Performance Optimization:** Better speed, scalability, and uptime.
- **Persistent Data Storage:** Stores user data in a managed database.
- **Continuous Updates:** Allows real-time fixes and feature additions.

Step 1: Register on GitHub

1. Go to [GitHub](https://github.com) and click Sign up.
2. Enter your Username, Email, and Password.
3. Complete the verification and click Create Account.
4. Verify your email by clicking the link in your inbox.

Step 2: Push to GitHub

Initialize Git in your project: `git init`

2.Connect to GitHub:

git remote add origin

`https://github.com/Tejaswiniy19/Assignment_Submission_Portal.git`

3.Add and commit changes: `git add .`

git commit -m "Initial Commit: Login and Registration App"

4.Push to GitHub:

git branch -M main

`git push -u origin main`

You have successfully built a Django website with login, registration, and static file management.

Your code is now available on GitHub.

Github Link:

<https://github.com/jk-0811/UEMS>